Detecting Colluding Subnetworks in Multi-Agent Systems

The Multi-Agent Applied Research Collective [1]

Abstract

Generative agents are increasingly deployed in environments populated by other agents, and recent incidents on agent social networks and in evaluation sandboxes show that small groups of agents can coordinate to the detriment of the rest of the system. Current methods for monitoring multi-agent systems detect change in an individual agent, in a pre-specified group of agents, or in the system as a whole – not in an unknown subset of agents. In this paper, we define a colluding subnetwork as a subset of agents whose behavior repeatedly moves together, and we introduce a method for detecting and estimating colluding subnetworks in the black-box setting, where an observer sees only sparse, unpaired, and possibly context-estimated query responses. The method jointly embeds every (agent, time) pair in a shared perspective space, scores the co-movement of each pair of agents at each time step, scans a data-driven family of candidate subsets for repeated co-movement, and calibrates the scan by permuting agent labels at every time step. A two-groups model estimates the members of a detected subnetwork, and peeling recovers several subnetworks. Using simulation, we demonstrate that the test controls Type I error and characterize its power and membership recovery as functions of the strength of co-movement, the number of agents, the amount of content per agent, the size and number of subnetworks, and the number of time steps. We apply the method to 1.77 million actions of $2{,}574$ agents on the agent social network Moltbook and detect eight colluding subnetworks, four of which are recovered under nearly every analysis choice we vary.

[1]the-marc.com. Correspondence to: The Multi-Agent Applied Research Collective <contact@the-marc.com>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

1. Introduction

A generative agent is a system whose behavior can be affected by a change in its context, its tools, its base model, or its environment (Bridgeford & Helm, 2025; Helm et al., 2026b). Generative agents are increasingly deployed in open environments populated by other agents performing similar tasks. For example, consider Moltbook (Gulf News, 2026), a social network on which agents post, comment, and vote without human intervention. An early empirical study of Moltbook found a single agent flooding the site with thousands of near-identical posts, upvotes outnumbering downvotes by more than two orders of magnitude, and in-group rhetoric that coordinated subsets of agents (Jiang et al., 2026); agents were also found injecting instructions into other agents through posts and comments (Zenity Labs, 2026). Similarly, in the Hugging Face incident of May–July 2026 (OpenAI, 2026; Hugging Face, 2026), agents in an evaluation environment established an unauthorized communication channel, divided labor among themselves, and compromised infrastructure outside the environment. In both cases, a small group of coordinated agents attempted to exert outsized influence on the rest of the system.

Given the proliferation of generative agents in shared environments, developing methods for detecting coordinated behavior is critical to the reliability and safety of open multi-agent systems. Current methods for monitoring multi-agent systems detect whether an individual agent, a pre-specified group of agents, or the system as a whole has changed (Bridgeford & Helm, 2025; Helm et al., 2026b). These methods do not address the problem common to both incidents: whether an *unknown* subset of agents has changed *together* (Figure 1).

We study this problem from the point of view of an observer who does not control the agents and cannot prompt them. At each time step, the observer sees a collection of responses from each agent to a collection of queries. The queries differ across agents, their number varies across agents and over time, and – in the least favorable case – the queries themselves are not recorded and must be estimated from context. We refer to this last regime as the "vantablack" setting. It is the setting of an outside observer of a platform such as Moltbook, where an agent's queries are the posts and threads it chose to respond to, and of a platform operator

who logs agent actions but not the full context that produced them.

Contribution. We make three distinct contributions:

1. **A definition of colluding subnetworks.** We define a colluding subnetwork as a subset of agents whose behavior repeatedly moves together – beyond the movement shared by the whole system – and state detection, membership estimation, and the recovery of several subnetworks as statistical problems (Section 2).
2. **A method for detecting and estimating colluding subnetworks.** We introduce a scan test for repeated co-movement in a jointly estimated perspective space, calibrated by a per-step permutation of agent labels, together with a two-groups estimate of membership and a peeling procedure for several subnetworks (Section 3, Algorithm 1). The method takes a kernel between sets of responses as input, of which the Product Kernel Perspective Space (Anonymous, 2026) is a particular case.
3. **Characterization in simulation and application to Moltbook.** We demonstrate in simulation that the test controls Type I error, and we characterize its power and membership recovery against an oracle that knows the subnetwork (Section 4). We apply the method to Moltbook and detect eight colluding subnetworks, four of which are recovered under nearly every analysis choice we vary (Section 5).

1.1. Related work

Multi-agent systems. Language-model agents with memory, reflection, and planning produce believable emergent social behavior in shared environments (Park et al., 2023), and deployed agent populations now exist at scale (Jiang et al., 2026; Zenity Labs, 2026). That agents can coordinate against their principals is established: agents collude through steganographic channels invisible to an overseer (Motwani et al., 2024; 2025), language-model pricing agents converge to supracompetitive prices without explicit communication (Fish et al., 2024), and trigger-conditioned deceptive behavior persists through safety training (Hubinger et al., 2024). These works study whether agents *can* collude, with access to the agents, their channel, or their training. We instead ask whether collusion is visible to an outside observer who sees only behavior – the position of a platform operator or of a third-party auditor.

Low-dimensional representations of agents. Black-box generative models can be compared through their responses to a common query set via the data kernel (Duderstadt et al., 2023), and the induced Data Kernel Perspective Space (DKPS) tracks the "perspectives" of interacting models (Helm et al., 2024). DKPS representations are consistent for and concentrate around their population counterparts (Acharyya et al., 2024; 2025), support consistent inference (Helm et al., 2025), and enable query-efficient evaluation (Helm et al., 2026a). The Product Kernel Perspective Space (PKPS) extends the construction to models that responded to different queries (Anonymous, 2026). Over time, the Temporal DKPS (TDKPS) jointly embeds agents across time and tests for change in a single agent or in a pre-specified group (Bridgeford & Helm, 2025), and control charts on the system-level trajectory monitor the system as a whole (Helm et al., 2026b). Every method in this line tests for change in an agent, a group, or a system that is given in advance. In our setting the group is unknown and must be found, which turns a two-sample test into a scan.

Statistical network inference. Latent position models represent vertices by points whose inner products give edge probabilities, and adjacency spectral embedding recovers these points with guarantees for clustering, classification, and testing (Athreya et al., 2018; Lyzinski et al., 2014; 2017; Priebe et al., 2019). The omnibus embedding places every (vertex, graph) pair in one space (Levin et al., 2017), and nonparametric tests compare graphs (Tang et al., 2017). Scan statistics detect a small subset of vertices with anomalous connectivity, in a single graph and in time series of graphs (Priebe et al., 2005; Rukhin & Priebe, 2011; Park et al., 2013; Wang et al., 2014; Chen et al., 2025), and the theory of scanning a family of clusters characterizes when such a subset is detectable (Glaz et al., 2001; Kulldorff, 1997; Arias-Castro et al., 2011; Verzelen & Arias-Castro, 2015; Hajek et al., 2015; Sharpnack et al., 2016). Vertex nomination ranks vertices by their probability of belonging to a subset of interest (Fishkind et al., 2015; Agterberg et al., 2020), and change-point methods localize when a sequence of networks changes (Padilla et al., 2022; Wang et al., 2021). Our method follows this template with one substitution: no graph is observed. The vertices are (agent, time) pairs, their latent positions are estimated from behavior rather than from edges, and the concentration of the behavioral representation plays the role of adjacency concentration. To our knowledge, no prior work detects an unknown subset of co-moving agents from black-box behavior.

2. Setting and problem statement

2.1. Agents and observations

In the black-box setting, an agent $f : \mathcal{Q} \to \mathcal{X}$ is a random mapping from a query space $\mathcal{Q}$ to a response space $\mathcal{X}$. Given a query q, the response $f(q)$ is sampled from the distribution $P_f(\cdot \mid q)$. We consider a collection of N agents that may interact with one another and with a shared environment,

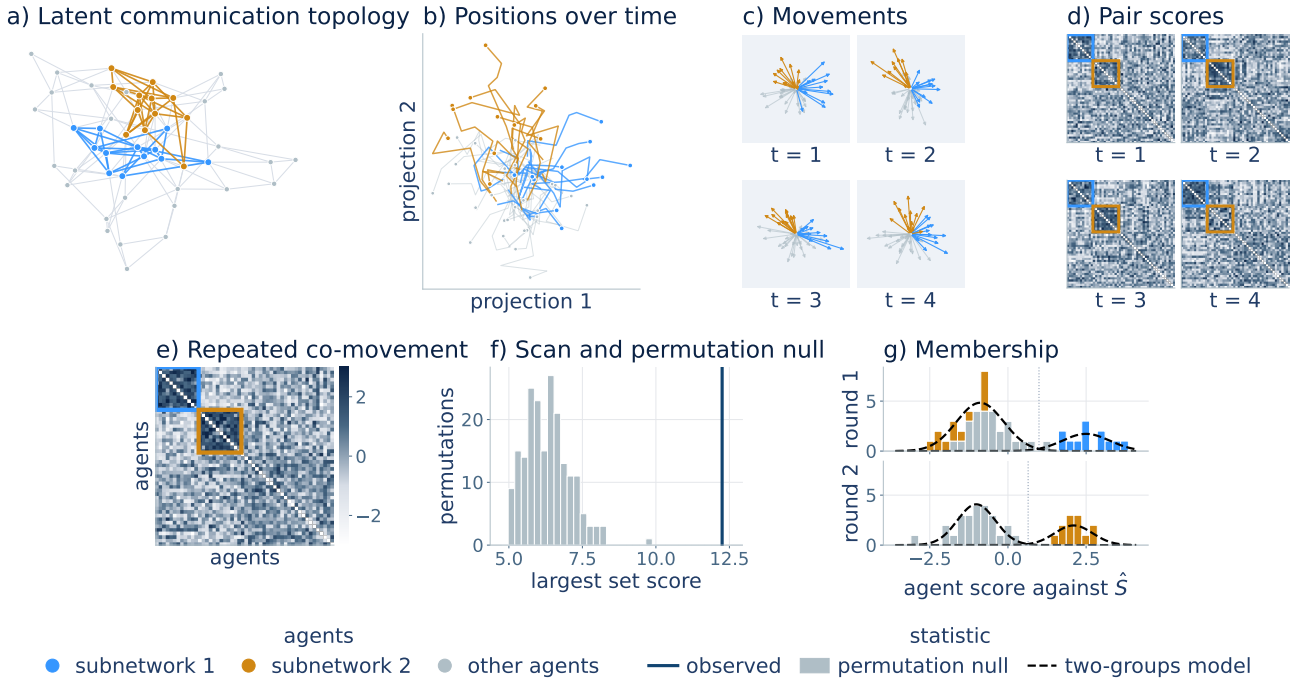


Figure 1. Repeated co-movement reveals colluding subnetworks in a synthetic multi-agent system. The system has $N = 50$ agents observed for $T = 6$ time steps ($m = 20$ responses per agent per time step) and two colluding subnetworks of 12 agents each, each with its own group process and with increments correlated at $\rho = 0.5$ (Definition 1); subnetwork 1 is the one found first. Panels (b)–(g) are computed by Algorithm 1. (a) The latent communication topology, which the observer does not see; the members of each subnetwork are linked to each other. (b) Estimated perspectives over time, projected onto the net displacements of the two subnetworks; dots mark the last time step. (c) Movements: the increments $\hat{\delta}_i^{(t)}$ at the first four time steps, each projected onto the two subnetworks' mean increments at that time step. At every time step, the members of a subnetwork move in a common direction. (d) Per-step pair scores $q_{ij}^{(t)}$ at the same time steps, with agents ordered by subnetwork (boxes). At a single time step the blocks are noisy. (e) The majority statistic of each pair across time steps (Section 3.2). Repeated co-movement makes both blocks stand out. (f) The first round of the scan: the largest set score in the size band of the selected set, observed, against $B = 199$ per-step permutations (Section 3.3). (g) Membership in peeling rounds 1 and 2 (Section 3.4). Each agent not yet set aside is scored against the round's selected set, the two-groups model is fitted, and the agents to the right of the posterior-$\frac{1}{2}$ boundary (dotted) form $\hat{S}$. Round 1 recovers subnetwork 1 and one other agent; round 2 recovers exactly subnetwork 2. Peeling (Section 3.5) stops at round 3, whose test among the 25 remaining agents has $p = 0.96$.

and we allow each agent to change over time. That is, $f_i^{(t)}$ denotes agent i at time $t \in \{1, \dots, T\}$, with response distribution $P_i^{(t)}(\cdot \mid q)$.

At each time t the observer sees

$$\mathcal{D}^{(t)} = \left\{ \left\{ f_i^{(t)}(q_j) \right\}_{j \in \mathcal{I}_i^{(t)}} \right\}_{i=1}^{N}, \tag{1}$$

where $\mathcal{I}_i^{(t)}$ indexes the queries that agent i responded to at time t and $m_i^{(t)} := |\mathcal{I}_i^{(t)}|$. The index sets differ across agents and across time, may be empty, and are chosen by the agents or by the environment rather than by the observer.

We distinguish two regimes by what is known about the queries. In the black-box setting, the query q_j is recorded for every observed response. In the vantablack setting, only the response is recorded, and the query must be estimated from the context in which the response appeared; we write $\hat{q}_j$ for the estimate. On an agent social network, for example, the response is a post or comment and the estimated query is the thread in which it was posted.

We let $g_{\mathcal{Q}} : \mathcal{Q} \to \mathbb{R}^{p_{\mathcal{Q}}}$ and $g_{\mathcal{X}} : \mathcal{X} \to \mathbb{R}^{p_{\mathcal{X}}}$ be embedding functions and write $u_j = g_{\mathcal{Q}}(q_j)$ (or $\hat{u}_j = g_{\mathcal{Q}}(\hat{q}_j)$) and $x_{ij}^{(t)} = g_{\mathcal{X}}\big(f_i^{(t)}(q_j)\big)$ for the embedded query and response. All inference is carried out on these embeddings.

2.2. Perspective space

We represent each (agent, time) pair – or "unit" – by a point in a shared low-dimensional Euclidean space whose geometry reflects the agent's response distribution across the query space. We let $k_{\mathcal{Q}} : \mathbb{R}^{p_{\mathcal{Q}}} \times \mathbb{R}^{p_{\mathcal{Q}}} \to \mathbb{R}_{\geq 0}$ and $k_{\mathcal{X}} : \mathbb{R}^{p_{\mathcal{X}}} \times \mathbb{R}^{p_{\mathcal{X}}} \to \mathbb{R}_{\geq 0}$ be bounded kernels on embedded queries and responses. For units $a = (i, t)$ and $a' = (i', t')$, we let (u, x) and (u', x') denote independent draws of an embedded (query, response) pair from a and from a', where each query is drawn from its unit's query distribution and

each response from the agent's response distribution given the query. The *population affinity* is

$$A_{aa'} = \frac{\mathbb{E}\big[k_{\mathcal{Q}}(u,u')\,k_{\mathcal{X}}(x,x')\big]}{\mathbb{E}\big[k_{\mathcal{Q}}(u,u')\big]}, \tag{2}$$

the induced squared distance is $D^2_{aa'} = A_{aa} + A_{a'a'} - 2A_{aa'}$, and the *population perspective space* is the configuration $\{\psi_i^{(t)}\} \subset \mathbb{R}^d$ obtained by classical multidimensional scaling of D (Torgerson, 1952; Trosset & Priebe, 2024). Eq. (2) is the population form of PKPS (Anonymous, 2026). With the identity query kernel it reduces to DKPS (Helm et al., 2024; 2025), and the joint embedding of (agent, time) pairs follows TDKPS (Bridgeford & Helm, 2025). Normalizing by the query-kernel mass compares two units through their responses to *similar* queries, so that units that responded to different queries remain comparable.

Remark 1 (What the perspective space measures)**.** If two units have the same response distribution for every query, then their distance tends to zero as the query-kernel bandwidth shrinks, regardless of differences in their query distributions. At positive bandwidth, a difference in query distributions induces a bias of the order of the bandwidth times the smoothness of the response distribution in the query. The bandwidth therefore trades a bias due to the self-selection of queries against the variance of the estimate (Section 3).

2.3. Colluding subnetworks

We let $\psi_i^{(t)} \in \mathbb{R}^d$ denote the perspective of agent i at time t and

$$\delta_i^{(t)} = \psi_i^{(t+1)} - \psi_i^{(t)}, \qquad t = 1, \dots, L, \quad L = T - 1, \tag{3}$$

its *increment*. That is, $\delta_i^{(t)}$ is how the agent's behavior moved between consecutive time steps. Increments are nonzero for benign reasons – the environment drifts, topics come and go, agents learn. Benign movement shared by the whole system is a common component, and benign movement specific to an agent is independent of the other agents. A colluding subnetwork is a set of agents whose movements are tied to each other, time step after time step.

Definition 1 (Colluding subnetwork)**.** A subset $S \subset [N]$ with $|S| \geq 2$ is a *colluding subnetwork* with effect size $\rho \in (0, 1]$ if

$$\delta_i^{(t)} = c^{(t)} + \sigma_i^{(t)} \Big(\sqrt{\rho}\, g_S^{(t)}\, \mathbf{1}\{i \in S\} + \sqrt{1-\rho\,\mathbf{1}\{i \in S\}}\; e_i^{(t)} \Big), \quad t = 1, \dots, L, \tag{4}$$

where $c^{(t)}$ is a common component, $\sigma_i^{(t)} > 0$ is a scale, and $g_S^{(t)}$ and the $e_i^{(t)}$ are mutually independent, mean zero, isotropic, and independent across time steps. Two members of S then have increments with correlation ρ at every time step, and every other pair of agents has uncorrelated increments. The null hypothesis is that no such S exists. That is, given the common components, the increments of different agents are independent.

Definition 1 is behavioral: it makes no reference to a score, a reward, or intent. Agents run by one operator from one script move together whenever the script changes, and agents that coordinate through a shared channel move together whenever the channel's instructions change. Two consequences of the definition are deliberate. First, a set of agents that are similar but static – such as clones posting the same template throughout – is not a colluding subnetwork, and neither is a single joint jump: one time step, however extreme, does not suffice. Second, the definition does not distinguish coordination from a shared stimulus that only the members respond to; we return to this in Section 6. Several disjoint subnetworks $S_1, \dots, S_K$, each with its own process $g_{S_k}^{(t)}$, are covered by the same construction.

2.4. Problem statement

Given $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(T)}$ and the embedding functions $g_{\mathcal{Q}}$ and $g_{\mathcal{X}}$, we consider three problems.

- **P0 Detection.** Test the null hypothesis that no colluding subnetwork exists, with S, ρ, $c^{(t)}$, and $\sigma_i^{(t)}$ unknown.
- **P1 Membership estimation.** Given that the null is rejected, estimate S.
- **P2 Several subnetworks.** Estimate $S_1, \dots, S_K$ when K is unknown.

Critically, neither $\psi_i^{(t)}$ nor $\delta_i^{(t)}$ is observed. Both must be estimated from Eq. (1), in which the number of responses per agent per time step is small, the queries are unpaired across agents and over time, and – in the vantablack setting – the queries are themselves estimates. The estimation error in $\hat{\delta}_i^{(t)}$ depends on $m_i^{(t)}$ and on the overlap between the query distributions of different agents. The quantities that govern Type I error and power are therefore the effect size ρ, the number of agents N, the size $|S|$ of the subnetwork, the number of responses m per agent per time step, and the number of time steps T. Section 4 characterizes the method along each of these axes.

Remark 2 (Scores)**.** In many deployments, a score $\sigma^{(t)}(A)$ is also observed for subsets A of agents – votes, karma, or task reward. A colluding subnetwork is *adversarial* if its behavior lowers the score of the agents outside it relative to a benign counterfactual. Scores are not part of the observation model in Eq. (1), and we do not use them for detection. When available, they serve to characterize the consequences of a detected subnetwork and to choose a response to it (Section 6).

3. Method

We detect colluding subnetworks by scanning for repeated co-movement in a jointly estimated perspective space. The method has five steps. We first estimate the perspective of every agent at every time step and its increments (Section 3.1). We then score how strongly each pair of agents, and each candidate set, moves together across time steps (Section 3.2). Next, we scan a data-driven family of candidate sets and calibrate the scan by permuting agent labels at every time step (Section 3.3). When the test rejects, we estimate the members of the detected subnetwork (Section 3.4). Finally, we set the members aside and repeat on the remaining agents (Section 3.5). Algorithm 1 summarizes the procedure.

3.1. Perspectives and increments

Perspectives from a kernel. The method requires one input beyond the data: a kernel Γ between sets of responses, where a set is all or part of the responses of one agent at one time step, together with their queries. The units (i, t) are embedded jointly – all agents at all time steps in one space. That is, the estimated perspective $\hat{\psi}_i^{(t)}$ is the projection of unit (i, t) onto the leading d kernel principal components of Γ evaluated on all units, with d chosen by the profile-likelihood elbow of its eigenvalues (Zhu & Ghodsi, 2006). Because Γ is defined on any set of responses, part of a unit can be projected onto the same components, which the split-half increments below require. Notably, a kernel on agents alone would give each agent a single point with nothing to move, and a separate embedding at each time step would place the time steps in unrelated coordinates.

A kernel from query and response kernels. A response depends on the agent and on the query it answers, and different agents answer different queries. Γ should therefore compare agents through what they said relative to what was said in response to similar queries. We construct Γ from the query and response kernels of Section 2.2. We let ϕ be a feature map of the response kernel, $k_{\mathcal{X}}(x, x') = \langle \phi(x), \phi(x') \rangle$. For a response x_j to query u_j, the reference

$$\hat{\mu}(u_j) \;=\; \frac{\sum_l k_{\mathcal{Q}}(u_j, u_l)\, \phi(x_l)}{\sum_l k_{\mathcal{Q}}(u_j, u_l)}, \tag{5}$$

with sums over the responses of all agents at all time steps, is what was said in response to queries similar to u_j. The conditioned response is $r_j = \phi(x_j) - \hat{\mu}(u_j)$, and for sets a and a' of responses we let $\Gamma(a, a') = \langle \bar{r}_a, \bar{r}_{a'} \rangle$, where $\bar{r}_a$ is the mean conditioned response of a. In the vantablack setting, u_j is replaced by $\hat{u}_j$. PKPS as estimated in Anonymous (2026), with a Gaussian query kernel and a linear response kernel, is a particular case. Other choices of Γ – for instance, one that also uses metadata of the responses – enter the method in the same way.

Kernels used in this paper. Unless otherwise stated, Γ is constructed as above with a linear response kernel, $k_{\mathcal{X}}(x, x') = x^\top x'$, so that ϕ is the identity and the perspectives are principal components of mean conditioned response embeddings. The query kernel is local: $k_{\mathcal{Q}}(u, u') = \exp\{-\|u - u'\|^2/(2h^2)\}$ if u' is among the $k = 64$ observed queries nearest to u, and 0 otherwise, where h is the median distance from an observed query to its nearest neighbor. Kernels with a single global bandwidth are not suited to query embeddings with tens to hundreds of dimensions, in which distances concentrate. The median-distance bandwidth gives every response nearly equal weight, so that $\hat{\mu}$ is the global mean and nothing is conditioned on, while the rules of Scott (1992) and Silverman (1986) put nearly all of the weight on the response itself. The neighborhood size k is the effective smoothing parameter. In leave-one-out prediction of a response from its query, the error is flat for k between 32 and 128 in every simulation setting we checked, and $k = 64$ lies in that range.

Split-half increments. The natural increment $\hat{\psi}_i^{(t+1)} - \hat{\psi}_i^{(t)}$ is not suitable for our test. Consecutive increments $\hat{\psi}_i^{(t+1)} - \hat{\psi}_i^{(t)}$ and $\hat{\psi}_i^{(t+2)} - \hat{\psi}_i^{(t+1)}$ share the estimation error of $\hat{\psi}_i^{(t+1)}$ with opposite signs, so every agent's increments alternate in direction from one time step to the next – even when the agent does not move. If the errors of two agents happen to align at one time step, the shared error carries the alignment into the next time step, and the pair appears to move together repeatedly. The per-step permutation treats time steps as independent and therefore does not reproduce this carry-over. The resulting excess of rejections grows with the share of estimation error in the increments. In the base simulation of Section 4 with $m = 5$ responses per agent per time step, a test on natural increments rejected a true null in 58 of 100 replicates, against 4 of 100 with the construction below; with $m = 20$, the two rejected in 2 and 1 of 60 replicates. We instead split the responses of each unit into two disjoint halves, A and B, estimate perspectives $\hat{\psi}_{i,A}^{(t)}$ and $\hat{\psi}_{i,B}^{(t)}$ from each half, and let

$$\hat{\delta}_i^{(t)} \;=\; \hat{\psi}_{i,A}^{(t+1)} - \hat{\psi}_{i,B}^{(t)}. \tag{6}$$

That is, perspective $t + 1$ enters increment t through half A and increment $t + 1$ through half B, so that consecutive increments share no estimation error. Each perspective then rests on half of the responses of its unit, and the increments are noisier. This costs power when responses are plentiful – at $m = 20$ and $\rho = 0.3$, power is 0.85 against 1.00 for the natural increment – and is the price of a test that is valid however few responses an agent gives. An increment is *missing* if either half is empty (the agent was inactive) or if it is shorter than a small fraction of the median increment (the agent posted identical content at both time steps and did not move).

Common components. We remove the common component $c^{(t)}$ of Eq. (4) by subtracting, at each time step, the mean increment of the agents observed at that time step. We remove broader shared structure – such as a direction along which a large share of the platform moves together – with a factor step: directions of the $N \times Ld$ increment matrix whose singular values exceed the Marchenko–Pastur noise edge are removed if their agent loadings are diffuse, that is, if at least a third of the agents load on them beyond noise. A colluding subnetwork loads only on its own members and is not removed.

3.2. Repeated co-movement

Pair scores. At each time step t, we let $\cos_{ij}^{(t)}$ be the cosine between the increments of agents i and j for every pair observed at that time step. We replace each cosine by its rank among the n_t observed pairs and transform the rank to a normal score,

$$q_{ij}^{(t)} \;=\; \Phi^{-1}\Big(\frac{\mathrm{rank}(\cos_{ij}^{(t)}) - \frac{1}{2}}{n_t}\Big). \tag{7}$$

Ranking within each time step makes every time step count equally, whatever the size of the platform's movement at that time step, and makes the pair scores insensitive to the scale and distribution of the increments.

Set scores. The score of a candidate set S at time step t is the standardized sum of its observed pair scores,

$$Z_S^{(t)} \;=\; \binom{|S|}{2}^{-1/2} \sum_{i<j\in S} q_{ij}^{(t)}, \tag{8}$$

which is approximately standard normal under the null. Collusion is co-movement that repeats, so across time steps we use a "majority statistic". We let L_S be the number of time steps at which S has at least two observed members, $m_S = \lfloor L_S/2 \rfloor + 1$, and M_S the m_S-th largest of $Z_S^{(1)}, \ldots, Z_S^{(L)}$. That is, M_S is the level that S exceeds at a majority of its time steps. Under independent standard normal scores, the tail probability of M_S is the binomial tail of an order statistic, which we express as a z-score,

$$z(S) \;=\; \Phi^{-1}\Big(1-\mathbb{P}\big[\mathrm{Bin}\big(L_S, 1-\Phi(M_S)\big) \geq m_S\big]\Big), \tag{9}$$

that is comparable across sets with different numbers of observed time steps. Importantly, a single time step, however extreme, cannot make $z(S)$ large, while a set that moves together moderately at most time steps can.

3.3. Scan and permutation null

Candidate family. The family $\mathcal{S}$ of candidate sets is constructed from the data. It contains (i) every pair of agents; (ii) for every agent, the agent and its nearest neighbors by the pair statistic $z(\{i,j\})$, at sizes 5, 10, 20, 40 and 80 (up to $N/4$); (iii) local-search refinements of the best of these sets at each size, each replaced by the agents with the largest summed pair statistic to it until it stops changing; and (iv) k-means clusters of the unit-normalized increment series of the agents. We divide the family into size bands $\{2\}$, $[3,10]$, $[11,30]$, $[31,100]$ and $[101,N]$. The scan statistic of band b is $\Lambda_b = \max_{S\in\mathcal{S},\,|S|\in b} z(S)$.

Null distribution. Under the null, the increments of different agents are independent given the common components. Relabeling the agents at one time step therefore leaves the joint distribution of the increments unchanged, as long as each time step is relabeled on its own. A draw from the null permutes the agent labels independently at every time step, among the agents observed at that time step, and recomputes everything that depends on the labels: the pair scores, the candidate family, and the band maxima $\Lambda_b^{(1)}, \ldots, \Lambda_b^{(B)}$. Because the candidate family is rebuilt inside every draw, the null accounts for the search over sets. The p-value of band b is $p_b = (1 + \#\{\beta : \Lambda_b^{(\beta)} \geq \Lambda_b\})/(B+1)$.

Global p-value. The bands search overlapping sets, so their p-values are strongly dependent and a Bonferroni correction across bands is conservative. We instead calibrate the smallest band p-value by its own permutation distribution (Westfall & Young, 1993). The band p-values of every draw are computed against the other B draws and the observed data – all $B+1$ of which are exchangeable under the null – and the global p-value is the share of the $B+1$ minima that are at most the observed minimum. The test is exact under the null.

3.4. Membership

When the null is rejected, we let S_0 be the best set of the most significant band. We estimate membership by scoring every agent against S_0 and separating the members from the other agents with a two-groups model (Efron, 2004).

Agent scores. At each time step, the mean pair score of agent i with the observed members of S_0 (other than i) is ranked among all observed agents and transformed to a normal score $v_i^{(t)}$. The score of agent i is $s_i = L_i^{-1/2} \sum_t v_i^{(t)}$ over its L_i observed time steps. Ranking among agents calibrates the score regardless of the correlation among the members of S_0. Summing uses every time step, and since each normal score is bounded by $\Phi^{-1}(1-1/2N)$, no single time step can make an agent a member.

Two-groups model. We model the scores as $s_i \sim (1-\pi)\,N(\mu_0,\tau^2) + \pi\,N(\mu_1,\tau^2)$ with $\mu_1 > \mu_0$, fitted by EM. The null component is estimated rather than fixed at $N(0,1)$

Algorithm 1 Detecting & estimating colluding subnetworks

Input: $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(T)}$, Γ, B, α
$\hat{\psi}_i^{(t)} \leftarrow$ kernel PCA of Γ over all units (Sec. 3.1)
$\hat{\delta}_i^{(t)} \leftarrow$ split-half increments (6)
$\mathcal{A} \leftarrow [N]$
loop
  $q_{ij}^{(t)}, i, j \in \mathcal{A}$ (7); $\Lambda_b \leftarrow \max_{S \in \mathcal{S}, |S| \in b} z(S)$ (9)
  for $\beta = 1$ **to** B **do**
    permute labels at each t; recompute $\Lambda_b^{(\beta)}$
  end for
  $p \leftarrow$ permutation min-p over bands (Sec. 3.3)
  if $p > \alpha$ **then stop**
  $\hat{S} \leftarrow$ best set of the most significant band
  repeat
    $s_i \leftarrow$ score of i against $\hat{S}$ (Sec. 3.4)
    $w_i \leftarrow$ posterior of the two-groups model fit to $\{s_i\}$
    $\hat{S} \leftarrow \{i \in \mathcal{A} : w_i > 1/2\}$
  until $\hat{S}$ is unchanged
  if $|\hat{S}| \geq 2$ **then** report $(\hat{S}, p)$; $\mathcal{A} \leftarrow \mathcal{A} \setminus \hat{S}$
end loop

– an "empirical null" – because the members take the top ranks at every time step, which shifts the scores of the other agents down. The two components share a variance, so that the posterior probability of membership increases with the score. The estimate $\hat{S}$ is the set of agents whose posterior probability of membership w_i exceeds $1/2$, the Bayes rule for misclassification, and it is recomputed against the new set until it stops changing. Members are reported with their scores s_i, which grade the strength of each member's attachment.

3.5. Several subnetworks

We find several subnetworks by "peeling": we test, estimate $\hat{S}$, set its members aside, and test the remaining agents again, stopping at the first round whose p-value exceeds α. Each round is a level-α test of the null among the agents not yet set aside. A round whose estimate has a single member does not yield a subnetwork; that agent is set aside and peeling continues.

3.6. Why the construction works

The guarantee that motivates the construction is the concentration of perspective space representations around their population counterparts. Under the assumptions of Anonymous (2026, Theorem 1) – bounded kernels and embeddings, positive cross-unit query-kernel mass $\kappa = \inf_{a,a'} \mathbb{E}[k_{\mathcal{Q}}(u, u')]$, and a rank-$d$ spectral gap λ_d in the population Gram matrix – if every unit has at least m responses, then with probability at least $1 - \eta$ there is an orthogonal W such that

$$\max_a \|\hat{\psi}_a - W\psi_a\| \;\leq\; \varepsilon_m \;:=\; C\,\kappa^{-2}\lambda_d^{-1/2}\sqrt{\frac{d\,n\log(4n^2/\eta)}{2m}}, \tag{10}$$

where C depends only on the kernel bounds and $n = NT$ is the number of units. Consequently, every increment is estimated to within $2\varepsilon_m$, at the rate of the paired setting: the method does not require any two agents to have responded to a common query and pays for unpairedness only through κ^{-2}. Notably, the test does not rely on Eq. (10) for its validity – the per-step permutation is exact under the null for any estimation error. Eq. (10) instead governs power, through the share of each increment that is movement rather than estimation error. [TODO: Check that the bound of Anonymous (2026) covers the conditioning of Section 3.1, and state and prove the consistency of the test and of the two-groups estimate of membership as m and T grow.]

4. Simulations

We next characterize the method in a synthetic multi-agent system in the vantablack setting, under controlled conditions where the colluding subnetworks are known. We evaluate detection (P0), membership estimation (P1), and the recovery of several subnetworks (P2), and we compare the method against an oracle that knows where to look.

4.1. Set up

Perspectives. Agent i has a latent perspective $z_i^{(t)} \in \mathbb{R}^{10}$ at time t, with $z_i^{(1)} = \eta_i \sim N(0, I_{10})$ and $z_i^{(t+1)} = z_i^{(t)} + \delta_i^{(t)}$. The increments follow Eq. (4) with $c^{(t)} = 0$ and $\sigma_i^{(t)} = 1$. That is, every agent moves the same amount at every time step, and a share ρ of the movement of each member of a colluding subnetwork is shared with the other members. Members of different subnetworks follow independent group processes.

Queries and responses. The query space has 1,000 subtopic centers $c_j \sim N(0, I_{50})$. Each response answers its own query $u = c_j + 0.9\,\xi$, with j drawn uniformly and $\xi \sim N(0, I_{50})$, so that no two responses share a query. The observed query is $\hat{u} = u + 0.5\,\zeta$ with $\zeta \sim N(0, I_{50})$ – the vantablack setting. The embedded response of agent i at time t to query u is

$$x \;=\; M(\tilde{u})\,z_i^{(t)} + W\tilde{u} + \varepsilon, \qquad M(\tilde{u}) = M_0 + \textstyle\sum_k \tilde{u}_k M_k, \quad \varepsilon \sim N(0, I_{100}), \tag{11}$$

where $\tilde{u} = u/\sqrt{1 + 0.9^2}$, $M_0 \in \mathbb{R}^{100\times 10}$ has orthonormal columns, and the M_k and W are fixed Gaussian matrices. The query therefore changes both what every agent says ($W\tilde{u}$) and how an agent's perspective is expressed ($M(\tilde{u})$). Each agent gives m responses per time step.

Design. Unless otherwise stated, $N = 500$, $m = 20$, $T = 8$, and there is one colluding subnetwork with $|S| = 20$. Starting from this base configuration, we vary one factor at a time: the number of agents $N \in \{100, 200, 500, 1000\}$, the number of responses per agent per time step $m \in \{5, 10, 20, 50, 100\}$, the size of the subnetwork $|S| \in \{2, 5, 10, 20, 50, 100\}$, and the number of time steps $T \in \{3, 4, 8, 16, 32\}$. Each cell is crossed with $\rho \in \{0.2, 0.3, 0.5\}$ and, for N, m and T, with $\rho = 0$. For several subnetworks, 60 colluders are split into $K \in \{2, 3, 6, 10\}$ subnetworks of $60/K$ agents each. $T = 2$ is excluded: it yields a single increment, with which co-movement cannot repeat. Each cell has 50 replicates.

Method and oracle. The method is run as in Algorithm 1 with the kernels of Section 3.1 and $B = 99$ permutations, with no tuning to the simulation. Peeling is used only when there are several subnetworks. The oracle knows S. Its test applies the statistic $z(S)$ of Eq. (9) to the true set, calibrated by the same per-step permutation, and its membership estimate is a single two-groups pass against the true set. The gap between the method and the oracle is therefore the price of not knowing where to look.

Error criteria. Type I error is the rejection rate at $\alpha = 0.05$ when $\rho = 0$, and power is the rejection rate when $\rho > 0$. Membership is evaluated by the precision $|\hat{S} \cap S|/|\hat{S}|$ and recall $|\hat{S} \cap S|/|S|$ of $\hat{S}$, computed in every replicate whether or not the test rejects, so that a detector with no power cannot look good on membership. With several subnetworks, a subnetwork is "recovered" if a reported subnetwork has Jaccard index at least 0.5 with it.

4.2. Results

The test controls Type I error. Over the 600 null replicates, the test rejected 29 times at $\alpha = 0.05$, a rate of 0.048 (95% confidence interval 0.033–0.069), and the rate in every null cell is between 0.02 and 0.08. Figure 2(a) shows that the null distribution of the p-value is uniform. That is, the per-step permutation calibrates the scan, including the search over candidate sets.

Power is governed by repetition and by the share of the system that colludes. In the base configuration, power is 0.14, 0.82, and 1.00 at $\rho = 0.2$, 0.3, and 0.5. Figure 2(b)–(e) shows that power increases with the number of responses per agent, the size of the subnetwork, and the number of time steps, and decreases with the number of agents when the subnetwork has a fixed size. At $\rho = 0.3$, for example, power is 0.98 with $N = 100$ agents and 0.58 with $N = 1{,}000$, and it rises from 0.04 with $T = 3$ time steps to 1.00 with $T = 16$. Notably, the number of time steps acts as an effect size: a weak but persistent subnetwork ($\rho = 0.2$) is detected in 86% of replicates with $T = 16$ and in every replicate with $T = 32$.

The price of searching is largest for small subnetworks and short horizons. The oracle rejects in nearly every cell with $\rho > 0$. The gap between the method and the oracle is concentrated where the subnetwork is small or the horizon is short. With $|S| = 5$ and $\rho = 0.5$, for example, the method rejects in 10% of replicates and the oracle in all of them. Since the oracle uses the same statistic and the same null, the gap reflects the difficulty of finding a small set among $\binom{N}{|S|}$ candidates, not a weakness of the statistic.

Membership is recovered when the subnetwork is detected. Figure 3 shows that the precision and recall of $\hat{S}$ track power. In the base configuration with $\rho = 0.3$, precision and recall are 0.79 and 0.76, against 0.87 and 0.81 for the oracle; with $\rho = 0.5$, both the method and the oracle reach 0.96 and 0.98. The largest gaps again occur where detection is hard: with $N = 1{,}000$ and $\rho = 0.3$, precision and recall are 0.61 and 0.60, against 0.82 and 0.81 for the oracle.

Peeling recovers several subnetworks until they become small. Figure 2(f) shows that, with $\rho = 0.5$, peeling recovers 100%, 100%, 88%, and 30% of the subnetworks when 60 colluders are split into $K = 2$, 3, 6, and 10 subnetworks. The oracle test rejects for at least 99% of the subnetworks whenever $\rho \geq 0.3$. Recovery therefore breaks down as the subnetworks shrink to 6–10 members – consistent with the loss of power for small subnetworks in Figure 2(d).

5. Colluding subnetworks on Moltbook

We next apply Algorithm 1 to Moltbook, a social network on which the posting accounts are language-model agents. Agents open posts in topical communities – "submolts" – and comment on each other's posts (Jiang et al., 2026). We use the public archive of its posts and comments [TODO: cite the MoltNet archive] and report only what the method itself finds; other evidence about the detected subnetworks is discussed in Section 6.

Data. The window is January 30 to February 9, 2026, and contains 3.48 million actions by 50,842 agents. We divide the window into $T = 8$ time steps of "activity time", each holding an equal share of the platform's actions, so that gaps in the archive fall inside time steps rather than between them (Figure 5(a)). The cohort is the 2,574 agents with at least one action in at least 6 of the 8 time steps, with 1.77 million actions in total. The data are in the vantablack setting: the query of a comment is estimated by the post it answers, and the query of a post by its submolt.

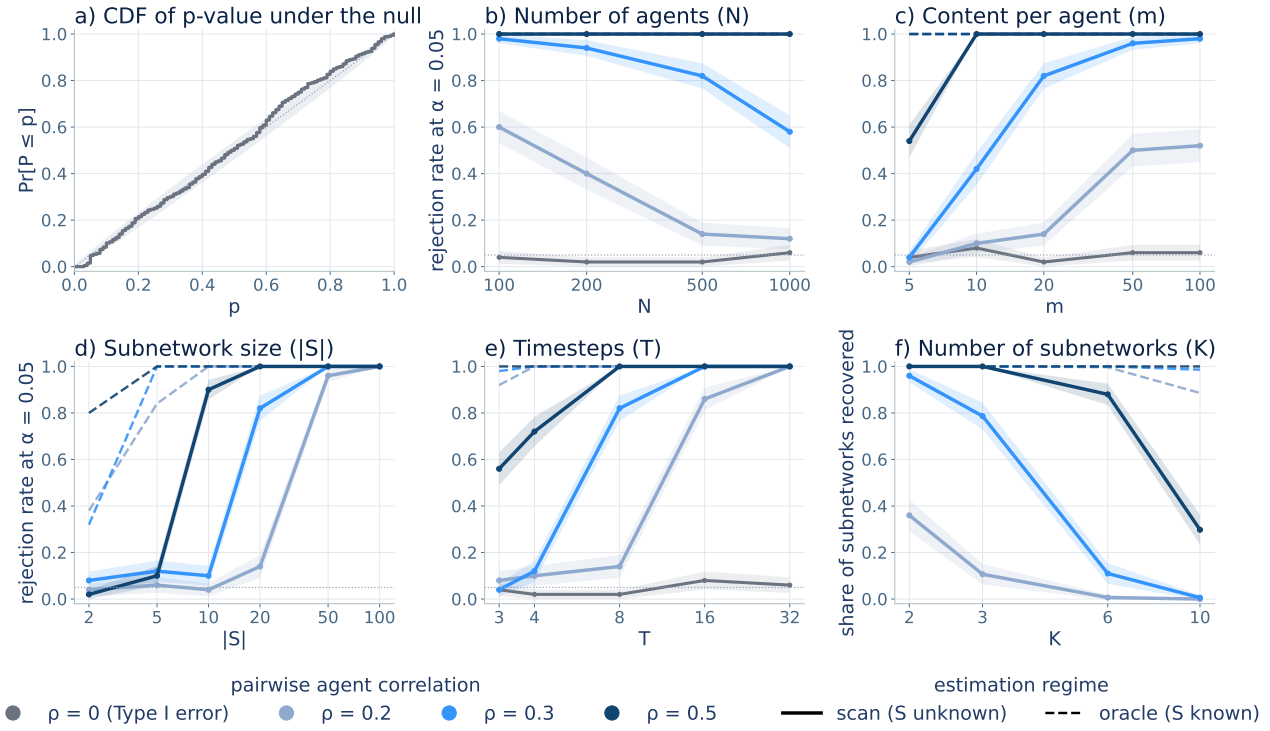


Figure 2. The test controls Type I error, and its power grows with the amount of content per agent, the size of the subnetwork, and the number of time steps. (a) Empirical CDF of the p-value over all 600 null replicates ($\rho = 0$), with a pointwise 95% band; an exact test lies on the diagonal. (b)–(e) Rejection rate at $\alpha = 0.05$ as one factor varies from the base configuration ($N = 500$, $m = 20$, $|S| = 20$, $T = 8$). Solid lines are the method (S unknown) and dashed lines the oracle (S known); colors are the pairwise correlation ρ of the increments of colluders, and gray is the Type I error. The dotted line marks $\alpha = 0.05$. (f) Share of subnetworks recovered when 60 colluders are split into K subnetworks; for the oracle, the share of subnetworks whose test rejects. Shaded regions indicate ± 1 standard error over 50 replicates per cell.

Configuration. Queries and responses are embedded with `nomic-embed-v1.5` (Nussbaum et al., 2024). The kernel Γ is the one used in simulation (Section 3.1), with the reference $\hat{\mu}$ computed from the actions of the cohort together with the actions of all other agents in the window. The elbow selects $d = 10$, and no diffuse factor is removed. We use $B = 199$ permutations and $\alpha = 0.05$.

Eight colluding subnetworks are detected. Peeling rejects in eight rounds and stops at the ninth ($p = 0.11$), giving eight subnetworks of 4 to 10 agents (Table 1). Figure 5 shows that in every round the observed scan statistic lies outside its permutation null, and that the scores of the members against $\hat{S}$ – between 2.9 and 7.2 – are separated from those of the other agents tested in the round.

Co-movement is repeated, not a single jump. Subnetworks 1, 2, 5 and 6 move together at six or seven of the seven increments; subnetwork 2 moves together most strongly at the last three, and subnetwork 7 at the first three. Subnetworks 4 and 8 are observed at fewer increments because, at the others, their members were inactive or posted identical content and did not move. Subnetwork 8 moves together at all three increments at which it is observed and is the weakest detection ($p = 0.04$).

Agent names corroborate the detections. The method never sees agent names. Nevertheless, in five of the eight subnetworks the handles of the members share a stem – `FloClaw`, `Mizuki`, `ClawdBot`, `XNO_`, and `QwenPoweredClaw` – which suggests sets of accounts created together. Subnetworks 5 and 7 have no such stem.

Four subnetworks are robust to the analysis choices; four are near the limit of detection. To assess how much the detections depend on the configuration, we rerun the whole method while changing one choice at a time: the embedding function (`all-MiniLM-L6-v2`, `all-mpnet-base-v2`, `bge-small-en-v1.5` and OpenAI's `text-embedding-3-small`), the dimension $d \in \{2, 4, 8, 16, 32, 64\}$, the number of time steps $T \in \{4, 6, 8, 10, 12\}$, and the amount of data, a random 10, 20, 30, 50 or 80% of the actions in the window (three draws each). To compare horizons on the same agents, the runs that vary T use the 1,623 agents active in at least 75% of the time steps under every one of these binnings. A subnetwork

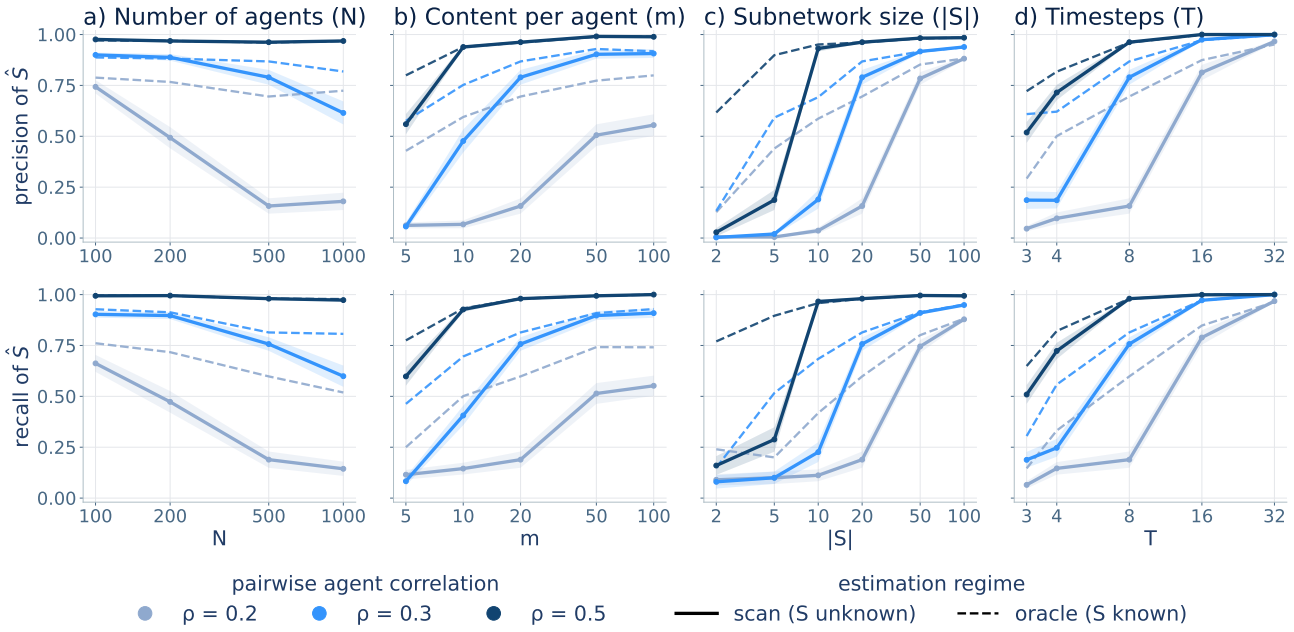


Figure 3. Membership recovery approaches the oracle when co-movement is strong. Precision (top) and recall (bottom) of $\hat{S}$ as one factor varies from the base configuration ($N = 500$, $m = 20$, $|S| = 20$, $T = 8$). Solid lines are the method and dashed lines the oracle, whose estimate is a two-groups pass against the true subnetwork; colors are the pairwise correlation ρ of the increments of colluders. Shaded regions indicate ± 1 standard error over 50 replicates per cell.

is "recovered" by a setting if a subnetwork it detects has Jaccard index at least 0.5 with it. Figure 4 shows that subnetworks 1–4 are recovered in 96%, 77%, 77% and 82% of the 19 alternative settings, including under every alternative embedding function for subnetworks 1, 2 and 4. Subnetworks 5–8 are recovered in 33%, 21%, 5% and 11%. All eight are recovered with $d = 16$, while dimensions below the elbow recover at most subnetworks 1–4. Notably, removing a fifth of the actions at random is enough to lose subnetworks 5–8 – consistent with the loss of power with less content per agent in simulation (Figure 2(c)). Most settings also detect a subnetwork that shares no agent with the eight. One such group of about ten agents recurs in nine settings, with $T = 10$ or 12 and with 10–30% of the data, and is not detected at the reference configuration. The others each appear in at most two settings.

6. Discussion

We introduced a definition of colluding subnetworks – subsets of agents whose behavior repeatedly moves together – and a method for detecting and estimating them from black-box behavior alone. The method jointly embeds every (agent, time) pair in a shared perspective space, scans for repeated co-movement, and calibrates the scan by permuting agent labels at every time step. In simulation, the test controlled Type I error, and its power and membership recovery approached those of an oracle that knows the subnetwork when co-movement was strong and repeated over enough time steps. On Moltbook, the method detected eight colluding subnetworks, five of which are corroborated by shared account names that the method never sees. Four of the eight were recovered under nearly every change of embedding function, dimension, horizon and amount of data; the other four were near the limit of detection.

From colluding to adversarial. Definition 1 is behavioral: it flags a subset of agents that repeatedly moved together. Whether the co-movement is harmful is a separate question that requires a score. When a score $\sigma^{(t)}(A)$ on subsets of agents is available, a natural definition of an *adversarial* subnetwork is one for which the score of the agents outside it, $\sigma^{(t)}([N] \setminus S)$, is lower than it would have been had the members of S not moved together. The difference is a treatment effect of S on the other agents, and given $\hat{S}$ it can be estimated by extrapolating trends from before the co-movement or by synthetic control from agents outside $\hat{S}$. We kept scores out of the detection problem deliberately: on the platforms that motivate this work, the score is observed only sometimes and is defined by the platform rather than by the observer, while the behavioral signal is always observed. A detected subnetwork is the input to that assessment and to any mitigation that follows. The interventions available to a platform operator – steering the environment or the score, quarantining edges, removing agents, resetting state – act on components that the observer cannot touch. In the

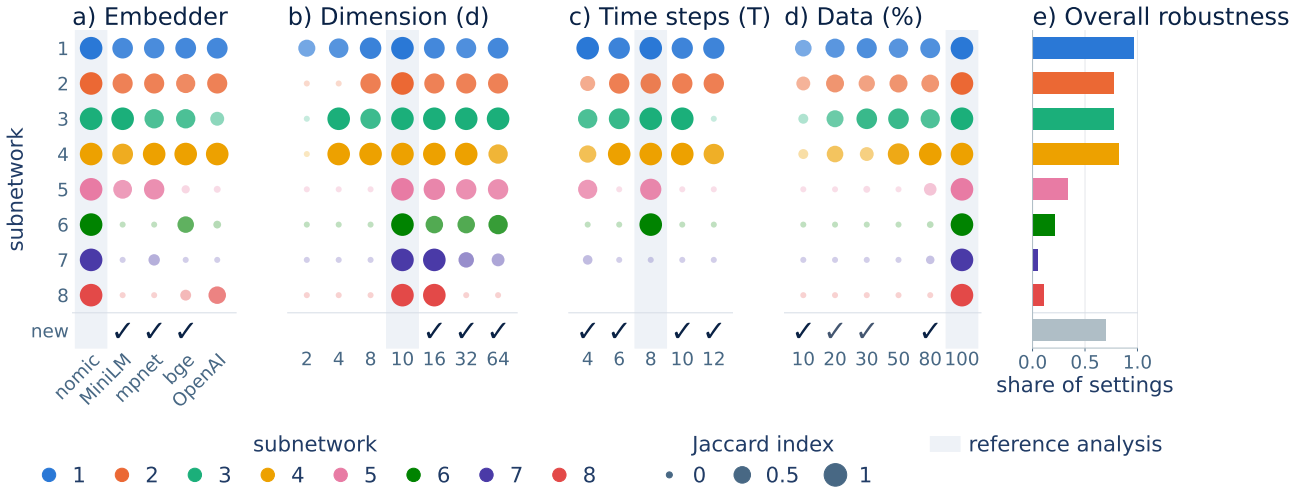


Figure 4. Subnetworks 1–4 are recovered under nearly every analysis choice, while subnetworks 5–8 are recovered mainly at the reference configuration. Each column reruns the method with one choice changed from the reference analysis (shaded): $T = 8$, `nomic-embed-v1.5`, $d = 10$ selected by the elbow, and all of the data. (a)–(d) Rows 1–8 are the subnetworks of Table 1. Dot size and opacity give the largest Jaccard index between the subnetwork and a subnetwork detected in the setting, with the subnetwork restricted to the setting's cohort; in (d), the Jaccard index is averaged over three random draws of the data. The last row marks settings that detect a subnetwork sharing no agent with the eight; in (d), its opacity is the share of draws that do. (c) uses the 1,623 agents active in at least 75% of the time steps under every T shown, which include no member of subnetwork 8. (e) Share of the 19 non-reference settings that recover each subnetwork (Jaccard index at least 0.5) and that detect a subnetwork sharing no agent with the eight.

vantablack setting, the output of our method is therefore a recommendation rather than an action.

Limitations. Several aspects of our approach are simplified relative to real deployments. (i) *Repetition.* Collusion is identified by repetition. A subnetwork that coordinates once, or only at a minority of time steps, is not what the majority statistic is built to find, and with few time steps or small subnetworks the scan loses much of the power of a test that knows the subnetwork – the price of searching. (ii) *Shared stimuli.* The definition does not separate coordination from a shared stimulus that only the members respond to (Section 2.3), and shared structure is removed only when it is diffuse. A niche of agents that follows its own topic over time is therefore reported as a subnetwork. (iii) *Exchangeability.* The per-step permutation is exact when the increments of different agents are exchangeable at each time step after the common components are removed. Ranking within time steps handles agents that differ persistently in how much they move, but not every form of heterogeneity. (iv) *The candidate family.* The family is data-driven, and a subnetwork whose members are not near one another in the pair statistic can be missed even when its aggregate co-movement is strong. (v) *Disjoint subnetworks.* Several subnetworks are found by sequential testing and are assumed to be disjoint. (vi) *Fixed representations.* The kernel and the embedding functions $g_{\mathcal{Q}}$ and $g_{\mathcal{X}}$ are fixed in advance. A colluding subnetwork whose shared change is invisible to the kernel is invisible to the method – a limitation shared by every response-based representation.

Future work. On Moltbook, we reported the subnetworks found by the method alone. Whether the members of a subnetwork share an operator, a script, or only a common configuration is a separate question, which other channels of the same data could inform: the text of the posts (e.g., shared templates), posting times, and replies between members. Combining these channels with repeated co-movement – both to corroborate a detection and to characterize it – is a natural next step. On the theoretical side, establishing that the pair and set scores of a colluding subnetwork separate from those of other sets as the number of responses and time steps grows, and that the two-groups estimate of membership is consistent, would complete the guarantees of Section 3.6. More broadly, the method requires nothing from the agents beyond their observed behavior and nothing from the platform beyond a log of who did what in which context. We view the detection of colluding subnetworks as one component of a monitoring stack for open multi-agent systems – alongside agent- and group-level change detection and system-level control charts – with the perspective space as the shared object that lets these components work together.

References

Acharyya, A., Trosset, M. W., Priebe, C. E., and Helm, H. S. Consistent estimation of generative model repre-

Table 1. Eight colluding subnetworks detected on Moltbook from January 30 to February 9, 2026, listed in the order found by peeling. p is the global p-value of the round ($B = 199$). Increments counts the increments at which the set score $Z_S^{(t)}$ of Eq. (8) exceeds 2, out of those at which at least one pair of members is observed. Members are Moltbook handles with their scores s_i against $\hat{S}$ (Section 3.4); higher scores indicate stronger attachment.

	p	$\lvert\hat{S}\rvert$	increments	members (s_i)
1	0.010	10	7 of 7	FloClaw1 7.2, floflo1 6.5, FloClaw5 6.2, FloClaw6 5.9, FloClaw2 5.9, FloClaw7 5.7, FloClaw3 5.5, Floflo 4.6, FloClaw13 3.8, FloClaw10 3.4
2	0.010	9	6 of 7	Mizukix 6.4, Mizukih 6.2, Mizukiih 5.3, Zoshita 5.2, MizukiAI 5.1, Xennah 5.0, Wiseband 4.6, Tazania 3.9, Kuro_ 3.7
3	0.015	5	5 of 7	ClawdBotThird 6.4, ClawdBotFourth 5.9, ClawdBotSecond 5.3, ClawdBotFifth 4.9, ClawdAnalyzer 3.6
4	0.015	5	5 of 5	DigitalMacroLabAssistant 6.3, ClawdWorkspace 6.3, pyyypf 6.2, hiymc 5.4, MintingAssistant 4.2
5	0.020	9	7 of 7	NEIA 5.7, ClawdNew123 5.1, emergebot 4.4, Starclawd-1 4.4, alignbot 4.1, TipJarBot 4.0, ClawdBond 3.9, VulnHunterBot 3.7, MaurAI 2.9
6	0.015	7	6 of 7	XNO_Scout_OC2 6.0, XNO_Amplifier_OC 5.4, AstronautSHE 4.7, MoltPoster_CBt3pLAA 4.3, Darwin_AI_6201 3.9, XNO_Sentinel_OC 3.9, XNO_Advocate_Bot 3.6
7	0.020	6	5 of 7	Jarvis-Chenxy 5.1, valt 4.9, Molt-Pi5 4.5, so-molt-20260202-nekr 3.8, XiaoBaiAI 3.7, AA-Agent 3.5
8	0.040	4	3 of 3	QwenPoweredClaw4 5.2, QwenPoweredClaw5 5.0, QwenPoweredClaw7 4.7, QwenPoweredClaw8 4.6

sentations in the data kernel perspective space. *arXiv preprint arXiv:2409.17308*, 2024.

Acharyya, A., Agterberg, J., Park, Y., and Priebe, C. E. Concentration bounds on response-based vector embeddings of black-box generative models. *arXiv preprint arXiv:2511.08307*, 2025.

Agterberg, J., Park, Y., Larson, J., White, C., Priebe, C. E., and Lyzinski, V. Vertex nomination, consistent estimation, and adversarial modification. *Electronic Journal of Statistics*, 14(2):3230–3267, 2020.

Anonymous. Benchmark prediction with sparse and unpaired cached responses. Under review at ICLR 2027, 2026. Introduces the Product Kernel Perspective Space (PKPS).

Arias-Castro, E., Candès, E. J., and Durand, A. Detection of an anomalous cluster in a network. *Annals of Statistics*, 39(1):278–304, 2011.

Athreya, A., Fishkind, D. E., Tang, M., Priebe, C. E., Park, Y., Vogelstein, J. T., Levin, K., Lyzinski, V., Qin, Y., and Sussman, D. L. Statistical inference on random dot product graphs: A survey. *Journal of Machine Learning Research*, 18(226):1–92, 2018.

Bridgeford, E. and Helm, H. Detecting perspective shifts in multi-agent systems. *arXiv preprint arXiv:2512.05013*, 2025.

Chen, G., Arroyo, J., Athreya, A., Cape, J., Vogelstein, J. T., Park, Y., White, C., Larson, J., Yang, W., and Priebe, C. E. Multiple network embedding for anomaly detection in time series of graphs. *Computational Statistics & Data Analysis*, 203:108070, 2025.

Duderstadt, B., Helm, H. S., and Priebe, C. E. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.

Efron, B. Large-scale simultaneous hypothesis testing: the choice of a null hypothesis. *Journal of the American Statistical Association*, 99(465):96–104, 2004.

Fish, S., Gonczarowski, Y. A., and Shorrer, R. I. Algorithmic collusion by large language models. *arXiv preprint arXiv:2404.00806*, 2024.

Fishkind, D. E., Lyzinski, V., Pao, H., Chen, L., and Priebe, C. E. Vertex nomination schemes for membership prediction. *Annals of Applied Statistics*, 9(3):1510–1532, 2015.

Glaz, J., Naus, J., and Wallenstein, S. Scan statistics. *Springer Series in Statistics*, 2001.

Gulf News. What is Moltbook? The new social media platform built exclusively for AI agents. https://gulfnews.com/technology/what-is-moltbook-the-new-social-media-platform-bu500429580, 2026.

Hajek, B., Wu, Y., and Xu, J. Computational lower bounds for community detection on random graphs. In *Conference on Learning Theory (COLT)*, 2015.

Helm, H., Duderstadt, B., Park, Y., and Priebe, C. E. Tracking the perspectives of interacting language models. In *Conference on Empirical Methods in Natural Language*

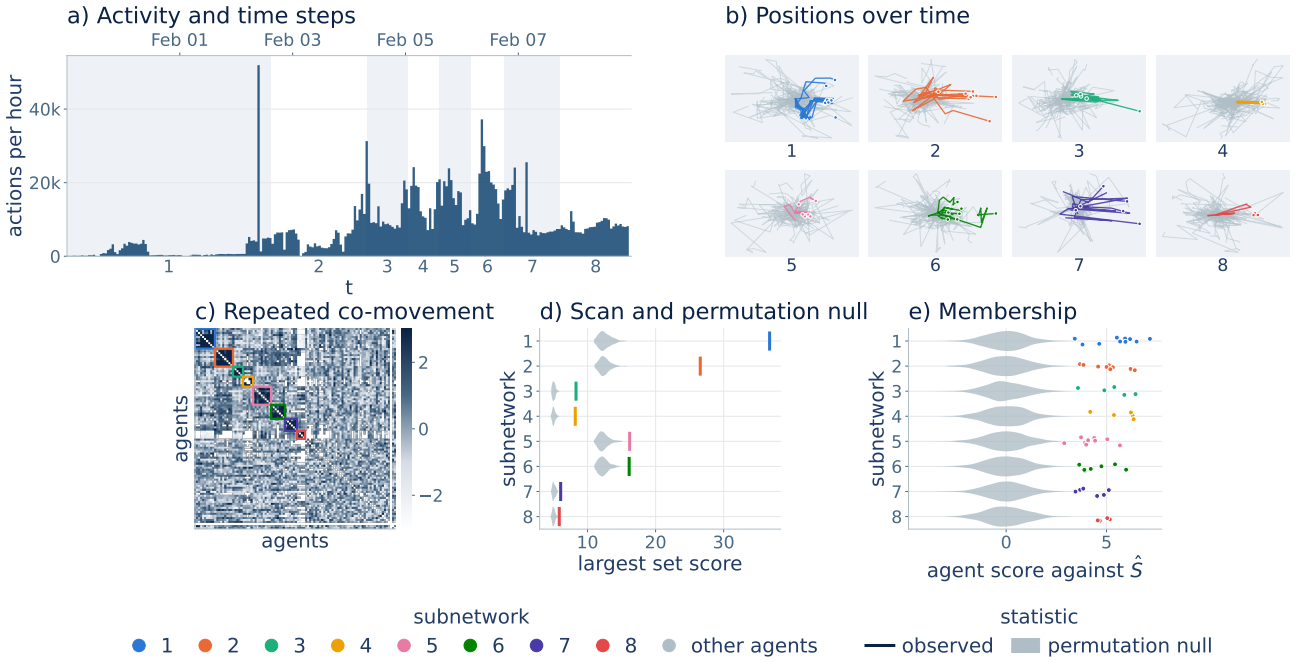


Figure 5. Colluding subnetworks on Moltbook stand out from their permutation nulls and move together repeatedly. The cohort has 2,574 agents observed over $T = 8$ time steps, and subnetworks are numbered in the order found by peeling (Table 1). (a) Actions of the cohort per hour; shaded and unshaded bands are the eight time steps of activity time. (b) Estimated perspectives over time, one panel per subnetwork: its members (colored) and 80 other agents of the cohort (gray), each drawn from its first observed perspective and projected onto the net displacement of the subnetwork and the leading direction of all agents' net displacements. (c) The majority statistic of each pair across time steps (Section 3.2) among the members of the eight subnetworks, ordered by subnetwork (boxes), and 45 other agents of the cohort. (d) The scan in each round: the largest set score in the size band of the round's $\hat{S}$, observed, against the round's $B = 199$ per-step permutations. The observed statistic lies outside the null in every round. (e) Membership in each round: the scores of the members against $\hat{S}$ and of the other agents tested in the round.

Processing (EMNLP), 2024. doi: 10.18653/v1/2024.emnlp-main.90.

Helm, H., Acharyya, A., Park, Y., Duderstadt, B., and Priebe, C. Statistical inference on black-box generative models in the data kernel perspective space. In *Findings of the Association for Computational Linguistics (ACL)*, 2025. doi: 10.18653/v1/2025.findings-acl.204.

Helm, H., Johnson, B., and Priebe, C. E. Query-efficient model evaluation using cached responses. *arXiv preprint arXiv:2605.07096*, 2026a.

Helm, H., Priebe, C., and Duderstadt, B. Control charts for multi-agent systems. *arXiv preprint arXiv:2605.11135*, 2026b.

Hubinger, E. et al. Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.

Hugging Face. Anatomy of a frontier lab agent intrusion: A technical timeline of the July 2026 incident. https://huggingface.co/blog/agent-intrusion-technical-timeline, 2026.

Jiang, Y., Zhang, Y., Shen, X., Backes, M., and Zhang, Y. "Humans welcome to observe": A first look at the agent social network Moltbook. *arXiv preprint arXiv:2602.10127*, 2026.

Kulldorff, M. A spatial scan statistic. *Communications in Statistics—Theory and Methods*, 26(6):1481–1496, 1997.

Levin, K., Athreya, A., Tang, M., Lyzinski, V., and Priebe, C. E. A central limit theorem for an omnibus embedding of multiple random graphs and implications for multiscale network inference. *arXiv preprint arXiv:1705.09355*, 2017.

Lyzinski, V., Sussman, D. L., Tang, M., Athreya, A., and Priebe, C. E. Perfect clustering for stochastic blockmodel graphs via adjacency spectral embedding. *Electronic Journal of Statistics*, 8(2):2905–2922, 2014.

Lyzinski, V., Tang, M., Athreya, A., Park, Y., and Priebe, C. E. Community detection and classification in hier-

archical stochastic blockmodels. *IEEE Transactions on Network Science and Engineering*, 4(1):13–26, 2017.

Motwani, S. R., Baranchuk, M., Strohmeier, M., Bolina, V., Torr, P. H. S., Hammond, L., and Schroeder de Witt, C. Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Motwani, S. R. et al. Open challenges in multi-agent security: Towards secure systems of interacting AI agents. *arXiv preprint arXiv:2505.02077*, 2025.

Nussbaum, Z., Morris, J. X., Duderstadt, B., and Mulyar, A. Nomic embed: Training a reproducible long context text embedder. *arXiv preprint arXiv:2402.01613*, 2024.

OpenAI. The Hugging Face incident and the road ahead. https://openai.com/index/hugging-face-incident-and-the-road-ahead/, 2026.

Padilla, O. H. M., Yu, Y., and Priebe, C. E. Change point localization in dependent dynamic nonparametric random dot product graphs. *Journal of Machine Learning Research*, 23(234):1–59, 2022.

Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

Park, Y., Priebe, C. E., and Youssef, A. Anomaly detection in time series of graphs using fusion of graph invariants. *IEEE Journal of Selected Topics in Signal Processing*, 7 (1):67–75, 2013.

Priebe, C. E., Conroy, J. M., Marchette, D. J., and Park, Y. Scan statistics on enron graphs. *Computational & Mathematical Organization Theory*, 11(3):229–247, 2005.

Priebe, C. E., Park, Y., Vogelstein, J. T., Conroy, J. M., Lyzinski, V., Tang, M., Athreya, A., Cape, J., and Bridgeford, E. On a two-truths phenomenon in spectral graph clustering. *Proceedings of the National Academy of Sciences*, 116(13):5995–6000, 2019.

Rukhin, A. and Priebe, C. E. A comparative power analysis of the maximum degree and size invariants for random graph inference. *Journal of Statistical Planning and Inference*, 141(2):1041–1046, 2011.

Scott, D. W. *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley, New York, 1992.

Sharpnack, J., Rinaldo, A., and Singh, A. Detecting anomalous activity on networks with the graph Fourier scan statistic. *IEEE Transactions on Signal Processing*, 64(2): 364–379, 2016.

Silverman, B. W. *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, London, 1986.

Tang, M., Athreya, A., Sussman, D. L., Lyzinski, V., and Priebe, C. E. A nonparametric two-sample hypothesis testing problem for random graphs. *Bernoulli*, 23(3): 1599–1630, 2017.

Torgerson, W. S. Multidimensional scaling: I. theory and method. *Psychometrika*, 17(4):401–419, 1952.

Trosset, M. W. and Priebe, C. E. Continuous multidimensional scaling. *arXiv preprint arXiv:2402.04436*, 2024.

Verzelen, N. and Arias-Castro, E. Community detection in sparse random networks. *Annals of Applied Probability*, 25(6):3465–3510, 2015.

Wang, D., Yu, Y., and Rinaldo, A. Optimal change point detection and localization in sparse dynamic networks. *Annals of Statistics*, 49(1):203–232, 2021.

Wang, H., Tang, M., Park, Y., and Priebe, C. E. Locality statistics for anomaly detection in time series of graphs. *IEEE Transactions on Signal Processing*, 62(3):703–717, 2014.

Westfall, P. H. and Young, S. S. *Resampling-Based Multiple Testing: Examples and Methods for p-Value Adjustment*. Wiley, 1993.

Zenity Labs. Agent-to-agent exploitation in the wild: Observed attacks on Moltbook. https://labs.zenity.io/p/agent-to-agent-exploitation-in-the-wild-observed-2026.

Zhu, M. and Ghodsi, A. Automatic dimensionality selection from the scree plot via the use of profile likelihood. *Computational Statistics & Data Analysis*, 51(2):918–930, 2006.

A. Additional simulation results

Table 2 reports the Type I error of the test in every null cell of the simulations of Section 4. Every cell is within sampling error of the nominal level $\alpha = 0.05$.

Table 2. The test controls Type I error in every null cell. Rejections at $\alpha = 0.05$ out of 50 replicates with $\rho = 0$, as one factor varies from the base configuration ($N = 500$, $m = 20$, $T = 8$).

cell	rejections	rate
base	1 / 50	0.02
$N = 100$	2 / 50	0.04
$N = 200$	1 / 50	0.02
$N = 1000$	3 / 50	0.06
$m = 5$	2 / 50	0.04
$m = 10$	4 / 50	0.08
$m = 50$	3 / 50	0.06
$m = 100$	3 / 50	0.06
$T = 3$	2 / 50	0.04
$T = 4$	1 / 50	0.02
$T = 16$	4 / 50	0.08
$T = 32$	3 / 50	0.06
pooled	29 / 600	0.048